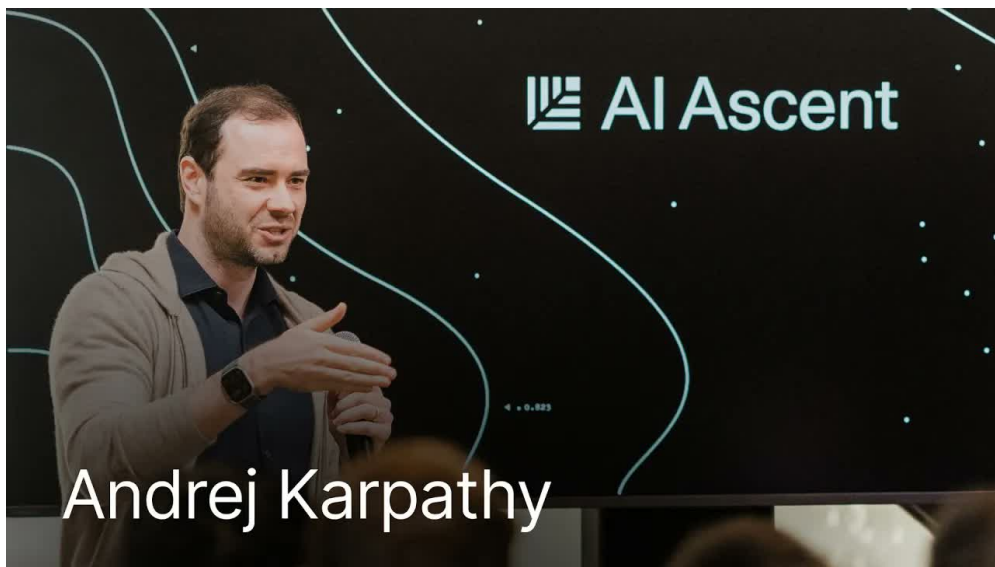


课程笔记

Andrej Karpathy: 从氛围编程到智能体工程

五道口纳什 & Codex

2026 年 5 月 10 日



视频作者/频道: Sequoia Capital

发布日期: 2026-04-29

视频时长: 29:49

视频链接: <https://www.youtube.com/watch?v=96jN20C0fLs>

目录

1 范式觉醒：从”落后感”到软件 3.0	4
1.1 开场：一个顶级程序员的”落后宣言”	4
1.2 转折点：2024 年 12 月的质变	4
1.3 软件 1.0→2.0→3.0：编程范式的三次跃迁	4
1.3.1 软件 1.0：显式规则编程	5
1.3.2 软件 2.0：神经网络权重编程	5
1.3.3 软件 3.0：提示词与上下文窗口编程	5
1.4 Agent 即安装器：OpenClaw 的启示	7
1.4.1 旧范式的困境	7
1.4.2 新范式的优雅	7
1.5 本章小结	8
2 范式对比：菜单生成 vs 原始提示	8
2.1 MenuGen 案例：旧范式下的”完整应用”	8
2.1.1 问题：看不懂的餐厅菜单	8
2.1.2 软件 1.0/2.0 思维的解决方案	9
2.2 软件 3.0 版本：一张照片 + 一句提示词	10
2.2.1 极简方案的诞生	10
2.2.2 核心差异的本质	11
2.3 新机会：不是加速旧事物，而是创造不可能	11
2.3.1 超越”编程加速”的框架	11
2.3.2 知识库：此前不可能存在的新事物	11
2.4 2026 年的 obvious：神经计算机的想象	12
2.4.1 从 MenuGen 到神经计算机	12
2.4.2 神经计算机的具体想象	12
2.4.3 历史的回响：计算器 vs 神经网络	13
2.5 本章小结	14
3 可验证性与锯齿形智能	14
3.1 可验证性框架：什么决定了 AI 的自动化速度	15
3.2 锯齿形智能：能重构 10 万行代码，却让你走路去洗车	16
3.3 数据分布的暴政：实验室在训练什么，你就得到什么	17
3.4 国际象棋的意外跃升：GPT-3.5 到 GPT-4 的启示	18
3.5 给创始人的建议：在可验证领域建立 RL 环境	19
3.6 一切终将可自动化——只是难易之分	20
3.7 本章小结	21
4 从氛围编程到智能体工程	22
4.1 Vibe Coding：抬高地板，让每个人都能写软件	22
4.2 Agentic Engineering：保住天花板，专业软件的质量底线	23
4.3 10 倍工程师？不，是 100 倍	24
4.4 AI 原生 vs 平庸使用者：工具投资的差距	24

4.5	招聘范式转变：从解谜题到做大项目	24
4.6	Agent 是”实习生”：有强大记忆力，但缺乏常识	25
4.7	品味与判断：人类仍然负责的核心能力	26
4.8	规格设计 (Spec) 的艺术：从 Plan Mode 到文档驱动	26
4.9	细节可以外包，原理必须掌握	27
4.10	为什么模型讨厌简化：RL 电路之外的挣扎	28
4.11	我们不是在建动物，我们在召唤幽灵	28
4.12	统计模拟电路：预训练 +RL 的叠加	29
4.13	没有内在动机，没有好奇心，没有恐惧	30
4.14	这种认知如何改变使用策略	30
4.15	本章小结	31
5	智能体无处不在：基础设施与教育	32
5.1	Agent 原生基础设施：一切为人类写的文档都是负担	33
5.1.1	问题：为什么今天的文档让 Agent 开发者抓狂	33
5.1.2	为什么简单方法不够：给 Agent 看人类文档是“二次翻译”	33
5.1.3	核心思想：Agent 原生 (Agent-Native) 的基础设施	33
5.1.4	如何工作：从人类文档到 Agent 接口	34
5.1.5	收获：基础设施范式的又一次迁移	34
5.2	传感器与执行器：分解世界的 Agent 视角	35
5.2.1	问题：Agent 如何感知和影响现实世界	35
5.2.2	为什么简单方法不够：人类中心的设计限制了自动化	35
5.2.3	核心思想：用传感器与执行器重新分解世界	35
5.2.4	如何工作：传感器-执行器循环	36
5.2.5	收获：一切系统都可以被 Agent 化	36
5.3	从部署地狱到一键部署：MenuGen 的教训	37
5.3.1	问题：写代码容易，部署代码难	37
5.3.2	为什么简单方法不够：现代部署流程充斥着人类操作	37
5.3.3	核心思想：部署应当是提示词级别的操作	37
5.3.4	如何工作：理想中的 Agent 部署流程	37
5.3.5	收获：MenuGen 是 Agent 原生基础设施的试金石	38
5.4	未来：我的 Agent 和你的 Agent 对话	39
5.4.1	问题：当每个人都有 Agent 时，协作方式会怎样改变	39
5.4.2	为什么简单方法不够：人类中介降低了协作效率	39
5.4.3	核心思想：Agent 作为个人和组织的代表	39
5.4.4	如何工作：Agent-to-Agent 协议	40
5.4.5	收获：从人机交互到机机交互	40
5.5	教育：可以外包思考，不能外包理解	41
5.5.1	问题：当智能变得廉价时，还有什么值得学	41
5.5.2	为什么简单方法不够：完全依赖 AI 会让人失去方向感	41
5.5.3	核心思想：理解是最后的护城河	41
5.5.4	如何工作：用 AI 增强理解，而非替代理解	42
5.5.5	收获：理解力是 AI 时代的元能力	42

5.6	本章小结	43
6	总结与延伸	44
6.1	核心论点回顾：三个范式、两层编程、一个未来	44
6.2	行动建议：今天就开始的 5 件事	44
6.3	开放问题：品味会自动化吗？理解会被替代吗？	45
6.4	终极测试	45

1 范式觉醒：从”落后感”到软件 3.0

1.1 开场：一个顶级程序员的”落后宣言”

2026 年初，在红杉资本（Sequoia Capital）主办的 AI Ascent 活动上，主持人向观众介绍了本场嘉宾——Andrej Karpathy。作为现代 AI 领域最具影响力的技术领袖之一，Karpathy 的履历堪称传奇：他是 OpenAI 的联合创始人之一，曾在特斯拉主导 Autopilot 项目，如今又创立了教育科技公司 Eureka Labs。然而，主持人抛出的开场问题却出人意料：就在几个月前，Karpathy 公开表示自己”从未像现在这样感觉作为一名程序员如此落后”。

对于一个站在技术金字塔顶端的人来说，这种”落后宣言”既令人震惊，又引人深思。Karpathy 坦言，这种感觉是”兴奋与不安的混合体”。要理解这句话的分量，我们需要先回顾他在过去一年中使用 AI 编码工具的经历。

在 2024 年的大部分时间里，Karpathy 和许多开发者一样，使用 Claude Code 等智能体工具辅助编程。当时的体验是：AI 能生成不错的代码片段，但时常出错，需要人工修正。这种模式下的 AI 更像是一个”有帮助但不可靠的助手”——它能加速部分工作，却无法让人完全放手。

然而，一切在 2024 年 12 月发生了质变。

1.2 转折点：2024 年 12 月的质变

Karpathy 将 2024 年 12 月描述为一个”清晰的转折点”。当时他正值假期，有了更多时间深入使用最新模型。他逐渐注意到一个微妙但根本性的变化：AI 生成的代码片段不再出错，连续多次请求后，输出依然稳定可靠。他发现自己已经记不清上一次手动修正 AI 代码是什么时候了——信任在不知不觉中建立起来。

为什么 2024 年 12 月是分水岭？

Karpathy 特别强调，许多人在 2024 年对 AI 的体验停留在”ChatGPT 级别的对话工具”，但如果你在那个时间点之后没有重新体验过最新的智能体工作流（agentic workflow），你就错过了根本性的变化。

关键区别：

- 之前：AI 生成代码块 → 人工检查 → 发现错误 → 手动修正
- 12 月之后：AI 生成代码块 → 直接信任 → 继续提出下一个需求 → AI 继续正确执行

这种从”需要修正”到”完全信任”的转变，标志着智能体工作流从”实验性工具”变成了”可靠的生产力引擎”。Karpathy 自己形容这种状态为”氛围编程”（vibe coding）——完全信任 AI 代理生成代码，不再手动修正，仅凭直觉和”氛围”驱动开发。

这个转折点让 Karpathy 一头扎进了”无限副业项目”的兔子洞。他的副业项目文件夹迅速膨胀，装满了各种随机实验——全部通过氛围编程完成。更重要的是，这个体验促使他开始系统性地思考这种新工作方式的深层含义和长远影响。

1.3 软件 1.0→2.0→3.0：编程范式的三次跃迁

要理解 2024 年 12 月究竟发生了什么，Karpathy 引入了一个强大的概念框架——软件范式的三次跃迁。

1.3.1 软件 1.0：显式规则编程

在软件 1.0 范式中，程序员直接编写代码，用显式的规则控制计算机行为。每一行代码都是人类意图的直接表达：

```
1 def is_even(n):
2     """显式编写判断逻辑"""
3     if n % 2 == 0:
4         return True
5     return False
```

Listing 1: 软件 1.0 示例：显式规则

这是传统编程的核心模式：人类理解问题 → 人类设计算法 → 人类编写代码 → 计算机执行。程序员对每一行代码拥有完全的控制权，也承担全部责任。

1.3.2 软件 2.0：神经网络权重编程

软件 2.0 的范式转变由深度学习带来。在这个范式中，程序员不再直接编写规则，而是通过创建数据集和训练神经网络来“编程”。权重（weights）被学习而非显式编写：

```
1 # 不再编写判断逻辑，而是准备数据和训练
2 import torch
3
4 # 数据集定义了"什么是偶数"
5 dataset = [(0,1), (1,0), (2,1), (3,0), ...]
6
7 # 神经网络架构定义了计算图
8 model = torch.nn.Sequential(
9     torch.nn.Linear(1, 10),
10    torch.nn.ReLU(),
11    torch.nn.Linear(10, 1),
12    torch.nn.Sigmoid()
13 )
14
15 # 训练过程"编程"了权重
16 optimizer = torch.optim.Adam(model.parameters())
17 # ... 训练循环
```

Listing 2: 软件 2.0 示例：数据集 + 训练

软件 2.0 的核心特征是：人类设计数据集、损失函数和网络架构，而具体的计算逻辑由梯度下降自动学习得到。Karpathy 曾在特斯拉大规模实践这一范式——自动驾驶系统的感知能力正是通过海量驾驶数据训练出来的。

1.3.3 软件 3.0：提示词与上下文窗口编程

软件 3.0 是 Karpathy 提出的最新范式，也是本章的核心论点。要理解它，我们需要先明确两个关键概念：

- **大语言模型 (LLM, Large Language Model)**：在海量文本上训练的大型神经网络，能够理解和生成自然语言。在软件 3.0 中，LLM 被比喻为一台可编程的计算机。

- **上下文窗口 (context window)**：LLM 能够同时处理的输入文本长度范围。在软件 3.0 中，上下文窗口是程序员控制 LLM 的主要杠杆。
- **提示词 (prompt)**：向 LLM 输入的指令或问题。在软件 3.0 中，提示词替代了传统代码的角色。



图 1: Andrej 阐述软件 1.0→2.0→3.0 的范式跃迁。

软件 3.0 的核心定义

软件 3.0 是以提示词和上下文窗口为杠杆，以 LLM 作为解释器执行计算的新兴编程范式。其核心等式可以概括为：

$$\text{程序}_{3.0} = \text{Prompt} + \text{Context Window} \xrightarrow{\text{LLM Interpreter}} \text{Computation}$$

其中：

- Prompt：提示词，即输入给 LLM 的指令或问题
- Context Window：上下文窗口，即 LLM 能同时处理的全部输入信息
- LLM Interpreter：大语言模型解释器，执行实际的”计算”
- Computation：在数字信息空间中完成的实际计算或推理结果

与传统编程的关键区别在于：你不再精确拼写每一条指令，而是通过提示词和上下文”引导”一个拥有自身智能的解释器完成目标。

Karpathy 指出，软件 3.0 的崛起有一个关键前提：当 LLM 在海量多样化的任务上训练时（尤其是通过训练互联网上的所有内容），它们隐式地学会了多任务处理。这使得 LLM 不再只是一个文本生成器，而是变成了一台真正意义上的”可编程计算机”——只是这台计算机的编程语言是自然语言，而你的”代码”就是提示词和上下文窗口中的内容。

⁰视频画面时间区间：00:02:20–00:02:35。

1.4 Agent 即安装器：OpenClaw 的启示

理论框架建立后，Karpathy 用一个具体例子来说明软件 3.0 思维方式的威力——OpenClaw 的安装过程。

1.4.1 旧范式的困境

在传统的软件 1.0 思维中，安装一个开发工具意味着运行一个 shell 脚本（bash script）。为了让安装程序适配各种不同的平台和计算机配置，这些脚本通常会急剧膨胀，变得极其复杂：

```
1 #!/bin/bash
2 # 传统安装脚本需要处理无数边界情况
3 check_os() { ... }
4 check_dependencies() { ... }
5 handle_mac_silicon() { ... }
6 handle_linux_distro() { ... }
7 # ... 数百行条件判断和错误处理
```

Listing 3: 传统安装脚本的复杂性

这种方式的本质问题是：你被困在软件 1.0 的宇宙中，试图用显式代码覆盖所有可能的场景。但现实世界的不确定性是无限的——不同操作系统版本、不同硬件配置、不同环境变量，都会导致脚本需要不断打补丁。

1.4.2 新范式的优雅

OpenClaw 的安装方式完全不同。它的“安装程序”不是一段代码，而是一段文本——一段你应该复制粘贴给你的 AI 智能体（agent）的文本：

```
1 # OpenClaw 的"安装程序"
2 # 复制以下内容，粘贴给你的 AI Agent：
3
4 "请帮我安装OpenClaw。检查我的系统环境，
5 处理任何依赖问题，并在遇到错误时自行调试。
6 完成后告诉我安装状态。"
```

Listing 4: 软件 3.0 的安装方式

这就是软件 3.0 范式的力量所在：

1. 你不再需要精确拼写所有细节——智能体拥有自己的“智能封装包”
2. 智能体观察你的环境——它主动查看你的计算机配置
3. 智能体执行智能操作——它根据具体情况做出判断
4. 智能体在循环中调试——遇到问题时自行解决

Agent（智能体/代理）的定义

Agent（智能体）是具有自主行动能力的 AI 系统，能在环境中感知、决策并执行动作，通常带有工具使用权限。

在软件 3.0 范式中，Agent 不再只是被动执行命令的工具，而是能够：

- 感知环境（读取文件系统、检测操作系统）
- 做出决策（选择正确的安装步骤）
- 执行动作（运行命令、修改配置）
- 自我修正（遇到错误时调整策略）

OpenClaw 的安装方式完美体现了这一转变：“安装程序”从一段死板的代码变成了一段给 Agent 的指令。

Karpathy 总结道，这种思维方式的转变极其深刻。现在的问题不再是“安装程序用什么语言写”，而是“应该复制粘贴什么文本给我的 Agent”。这就是软件 3.0 的编程范式。

1.5 本章小结

本章从 Andrej Karpathy 的“落后宣言”出发，梳理了编程范式从软件 1.0 到软件 3.0 的三次跃迁：

1. **软件 1.0**：人类显式编写规则代码，直接控制计算机行为
2. **软件 2.0**：人类通过数据集和训练“编程”神经网络，权重被学习而非显式编写
3. **软件 3.0**：人类通过提示词和上下文窗口“编程”LLM，让智能解释器执行计算

2024 年 12 月的质变标志着智能体工作流从“实验性工具”跃升为“可靠的生产力引擎”，使得氛围编程成为可能。而 OpenClaw 的安装示例则生动展示了软件 3.0 思维的核心优势：与其用显式代码穷举所有边界情况，不如将智能封装在 Agent 中，让它根据具体环境自适应地解决问题。

下一章将通过 Karpathy 亲身经历的 MenuGen 案例，对比旧范式下的“完整应用”与软件 3.0 范式下的极简方案，进一步揭示范式跃迁的深层含义。

2 范式对比：菜单生成 vs 原始提示

2.1 MenuGen 案例：旧范式下的“完整应用”

在理解了软件 3.0 的理论框架后，Karpathy 用一个亲身经历的案例来展示范式差异的真实威力——MenuGen（菜单生成）项目。

2.1.1 问题：看不懂的餐厅菜单

这个项目的灵感来自一个常见的日常困扰。Karpathy 提到，当他去餐厅时，通常有 30% 到 50% 的菜品名称是他完全不知道的——菜单上只有文字，没有图片。对于不熟悉该菜系的人来说，点一道“Bouillabaisse”或“Char Siu”就像开盲盒。

2.1.2 软件 1.0/2.0 思维的解决方案

在旧范式下，Karpathy 用氛围编程（vibe coding）构建了一个”完整”的 Web 应用来解决这个问题。这个应用的工作流程是：

1. 用户上传餐厅菜单照片
2. 应用使用 OCR（光学字符识别）提取所有菜品名称
3. 应用调用图像生成器，为每个菜品生成一张示意图片
4. 应用在 Vercel 上重新渲染整个菜单页面，展示菜品名称 + 生成图片



图 2: Andrej 回顾 MenuGen 旧范式开发的复杂架构。

从技术栈来看，这是一个典型的现代 Web 应用：

```
1 # MenuGen 旧范式架构
2 前端：Next.js + React（部署于 Vercel）
3 OCR：图像文字识别服务
4 图像生成：Stable Diffusion / DALL-E API
5 后端：处理上传、调用 OCR、调用图像生成、渲染页面
6 部署：Vercel + 手动配置 DNS、第三方服务、环境变量
```

Listing 5: MenuGen 旧范式技术栈

Karpathy 投入了大量精力构建这个应用——编写前后端代码、集成多个 API 服务、处理图像上传和渲染逻辑、部署到 Vercel 平台。从传统软件工程的角度看，这是一个功能完整、架构合理的解决方案。

然而，当他看到软件 3.0 版本的解决方案时，他意识到自己构建的整套应用都是**虚假的复杂性**（spurious complexity）。

⁰视频画面时间区间：00:04:50–00:05:05。

2.2 软件 3.0 版本：一张照片 + 一句提示词

不要在旧范式里思考”加速”

Karpathy 对 MenuGen 的反思揭示了一个常见误区：很多开发者在面对 AI 时，仍然用旧范式的思维来思考问题——”我如何用这个新工具更快地做我原来在做的事？”

但真正的范式跃迁不是**加速旧事物**，而是**让旧事物消失**。

MenuGen 的完整应用架构——OCR 模块、图像生成模块、Web 渲染层、部署管道——在软件 3.0 范式下，这些代码本不该存在。它们不是被 AI”加速”了，而是被 AI **吸收**了。

2.2.1 极简方案的诞生

软件 3.0 版本的 MenuGen 解决方案让 Karpathy”大吃一惊”。它不需要任何应用代码，不需要任何部署，只需要：

1. 一张菜单照片（原始输入）
2. 一句提示词：”用 Nanobanana 把菜品图片叠加到菜单上”

Gemini 模型接收这张照片和这句简短的指令，直接返回一张图片——这张图片就是原始菜单照片，但菜品位置已经被 AI 生成的食物图片精确填充。没有 OCR 模块，没有 Web 应用，没有部署流程。

```
1 # 输入：一张照片 + 一句提示词
2 photo = "menu_photo.jpg" # 餐厅菜单照片
3 prompt = "Use Nanobanana to overlay images of the
4          dishes onto this menu photo"
5
6 # 输出：一张直接可用的图片
7 result = gemini.generate(image=photo, prompt=prompt)
8 # result 就是渲染好的菜单图片，无需任何中间应用
```

Listing 6: 软件 3.0 版本的 MenuGen



图 3: raw prompt 带来的震撼效果——一张照片加一句提示词即可生成完整可视化菜单。

Karpathy 的原话是：“我的整个 MenuGen 都是虚假的 (spurious)。它在旧范式下工作，这个应用本不该存在。”

2.2.2 核心差异的本质

两个方案的根本区别在于信息流动的路径：

旧范式 (MenuGen 应用)：

照片 → OCR → 文本 → 图像生成 API → 图片集合 → Web 渲染 → 浏览器展示

软件 3.0 范式 (原始提示)：

照片 + 提示词 $\xrightarrow{\text{神经网络}}$ 输出图片

在软件 3.0 范式中，神经网络做了绝大部分工作。你的提示词或上下文就是输入（一张照片 + 一句指令），输出就是最终结果（一张图片），中间不需要任何应用层。

2.3 新机会：不是加速旧事物，而是创造不可能

MenuGen 的对比让 Karpathy 意识到了一个更深层的问题：很多人仍然在用旧范式思考 AI 的能力边界。

2.3.1 超越“编程加速”的框架

Karpathy 明确指出，这种思维方式本身就是“旧心态”。问题不只是“编程变得更快了”——实际上，这涉及更广泛的信息处理自动化：

“Previous code worked over structured data, right? And you write code over structured data. But... this is not even a program. This is not something that could exist before.”

传统代码处理的是结构化数据——数据库记录、API 响应、配置文件。但 LLM 能够处理的是非结构化信息——文档、图片、语音、自由文本。这意味着全新的应用类型成为可能。

2.3.2 知识库：此前不可能存在的新事物

Karpathy 举了一个更激进的例子——LLM 知识库 (knowledge base) 项目。这个项目的核心思想是：让 LLM 根据你的文档自动生成一个组织化的维基系统。

```
1 # 传统方式：不可能实现
2 # 没有代码能“理解”任意文档并生成有用的知识库
3
4 # 软件3.0方式：直接可行
5 documents = ["report1.pdf", "notes.txt", "meeting_transcript.md", ...]
6 prompt = """基于这些文档，为我创建一个结构化的知识库。
7 提取关键概念，建立关联，生成易于浏览的维基页面。"""
8
9 knowledge_base = llm.generate(documents=documents, prompt=prompt)
```

Listing 7: LLM 知识库：此前不可能的应用

⁰视频画面时间区间：00:05:35–00:05:50。

这不是一个“更快的旧事物”——在 LLM 出现之前，没有任何代码能够基于一堆事实文档自动创建一个有意义的知识库。这不是自动化了一个已有的流程，而是创造了一个此前不可能存在的全新事物。

新机会的核心洞察

Karpathy 反复强调的思维方式转变：

- 不要只问：“什么已有的东西可以做得更快？”
- 更要问：“什么以前不可能的事情，现在变得可能了？”

他认为后者“更令人兴奋”。软件 3.0 的真正潜力不在于替代现有的代码工作流，而在于解锁全新的信息处理范式。

2.4 2026 年的 obvious：神经计算机的想象

interviewer 追问了一个极具前瞻性的问题：如果 extrapolate 这个趋势，2026 年什么会变得“显而易见”（obvious）？就像 90 年代建网站、2010 年代做移动应用、过去十年做 SaaS 一样，下一个平台级机会是什么？

2.4.1 从 MenuGen 到神经计算机

Karpathy 的回答大胆而深刻。他认为 extrapolation 会指向一个“非常怪异和陌生”的方向——神经计算机（neural computer）。

未来架构：flipped 设计

Karpathy 预言了一种**翻转的架构**（flipped architecture）：

- **当前架构**：CPU/GPU 为主，神经网络作为软件运行在经典计算机上（虚拟化）
- **未来架构**：神经网络成为宿主进程（host process），CPU 退化为协处理器（co-processor）

这意味着：

未来计算机 = 神经网络核心 + CPU 协处理器 + 工具使用作为历史附肢

其中“工具使用”（tool use）被 Karpathy 比喻为“历史附肢”（historical appendage）——用于处理那些仍然需要确定性计算的任务。

2.4.2 神经计算机的具体想象

Karpathy 描述了他想象中的神经计算机：

1. **原始输入**：设备接收原始视频或音频流——没有键盘、没有鼠标、没有传统 UI
2. **神经网络处理**：输入直接进入神经网络核心进行处理和理解
3. **扩散模型渲染**：使用扩散模型（diffusion model）实时渲染用户界面——这个界面是**为当下这一刻独特生成的**



图 4: Andrej 展望神经计算机的未来——以神经网络为核心、CPU 为协处理器的翻转架构。

这种设备与传统计算机的根本区别在于：传统计算机的 UI 是预先设计好的固定界面（按钮、菜单、窗口），而神经计算机的 UI 是**动态生成**的——针对你当前的具体情境、具体需求，实时渲染出最合适的交互界面。

2.4.3 历史的回响：计算器 vs 神经网络

Karpathy 引用了一个鲜为人知的历史细节：在计算机早期（1950-60 年代），人们其实并不确定计算机应该长什么样。当时的两个 competing vision 是：

- **计算器路径**：精确、确定性的符号计算（我们最终选择的道路）
- **神经网络路径**：模拟生物大脑的模式识别和自适应处理

历史选择了计算器路径，我们在此基础上构建了经典计算体系，而神经网络目前只是作为软件虚拟化运行在这些经典计算机上。但 Karpathy 认为，这个关系可能会**翻转**——神经网络成为核心，经典 CPU 退化为处理确定性任务的协处理器。

```
1 # 神经计算机概念架构（推测性）
2
3 # 输入层：原始感知数据
4 raw_video = capture_camera_stream()
5 raw_audio = capture_microphone_stream()
6
7 # 核心：神经网络理解+推理
8 understanding = neural_core.process(
9     video=raw_video,
10    audio=raw_audio,
11    context=user_history
12 )
13
```

⁰ 视频画面时间区间：00:07:30-00:07:45。

```

14 # 输出层：扩散模型实时渲染UI
15 ui = diffusion_renderer.generate(
16     content=understanding,
17     style=context_appropriate_style,
18     interaction_mode=current_task
19 )
20
21 # 工具使用：仅在需要确定性计算时调用
22 deterministic_result = cpu_coprocessor.calculate(
23     tool=understanding.required_tool
24 )

```

Listing 8: 神经计算机的想象架构

关于扩散模型和工具使用

扩散模型（diffusion model）是一类生成式 AI 模型，通过逐步去噪生成图像。在 Karpathy 的设想中，它将成为未来神经计算机的 UI 渲染引擎——不是渲染预设计的组件，而是根据当前情境生成最合适的界面。

工具使用（tool use）是 LLM 调用外部工具（如计算器、代码执行器、API）的能力。在神经计算机的架构中，这些工具不再是核心能力，而是被比喻为“历史附肢”——用于处理那些神经网络本身不适合做的确定性任务。

Karpathy 坦承这个 extrapolation “看起来非常怪异和陌生”，但他认为这是合理的方向。他最后补充道：“这个 progression 还有待观察（TBD）。”

2.5 本章小结

本章通过 MenuGen 案例的深刻对比，揭示了软件 3.0 范式的核心洞察：

1. **旧范式的陷阱**：MenuGen“完整应用”代表了旧范式下的虚假复杂性——OCR、Web 渲染、部署管道等模块在软件 3.0 范式下本不该存在
2. **新范式的极简**：一张照片 + 一句提示词，神经网络直接完成端到端的任务，无需任何中间应用层
3. **新机会的维度**：不只是加速旧事物，而是创造此前不可能存在的新事物（如 LLM 自动知识库）
4. **未来的想象**：神经计算机可能翻转当前架构——神经网络成为核心宿主，CPU 退化为协处理器，扩散模型实时渲染情境化 UI

Karpathy 的核心警告是：不要在旧范式里思考“加速”。当你真正拥抱软件 3.0 时，很多代码和架构会消失——不是被优化了，而是被吸收了。

3 可验证性与锯齿形智能

时间范围：09:41 – 15:46

3.1 可验证性框架：什么决定了 AI 的自动化速度

为什么有些领域（如代码、数学）的 AI 自动化进展神速，而另一些领域却举步维艰？Andrej Karpathy 给出的核心答案是：**可验证性**（verifiability）——即输出结果能否被自动验证正确性。

传统计算机 vs LLM 自动化的边界

传统计算机自动化的前提是**可编码**（specifiable in code）：只要你能把规则写成代码，计算机就能执行。而 LLM 自动化的前提是**可验证**（verifiable）：只要你能自动判断输出是否正确，强化学习就能让模型在该领域飞速进步。

这一差异决定了两个时代的自动化边界：

- **软件 1.0 时代**：自动化的是**可编码**的任务——规则明确、逻辑清晰、能写成 if-else 的重复劳动。
- **软件 3.0 时代**：自动化的是**可验证**的任务——即使规则难以显式书写，只要能判断对错，LLM 就能通过试错学会。

前沿实验室在训练大模型时，本质上是在构建**巨型强化学习环境**（giant reinforcement learning environments）。模型生成输出后，环境会给出验证奖励（verification rewards），模型据此调整自身行为。这个过程的数学本质可以描述为：

$$\theta_{t+1} = \theta_t + \eta \cdot \nabla_{\theta} \mathbb{E}_{\pi_{\theta}} [R(s, a)]$$

其中：

- θ_t 表示模型在第 t 步的参数；
- η 是学习率；
- π_{θ} 是参数为 θ 的策略（即模型生成输出的方式）；
- $R(s, a)$ 是在状态 s 下采取动作 a 后获得的奖励信号；
- ∇_{θ} 表示对模型参数求梯度，以最大化期望奖励。

核心洞察：强化学习的效率高度依赖于奖励信号的**自动化程度**。在代码领域，编译器可以立即告诉你代码是否运行通过；在数学领域，符号计算系统可以自动验证推导是否正确。这些领域天然具备**高密度、低延迟的奖励信号**，因此 RL 可以大规模展开，模型能力随之飙升。



图 5: 可验证性框架的引入——决定 AI 自动化速度的核心变量。

3.2 锯齿形智能：能重构 10 万行代码，却让你走路去洗车

由于可验证性分布的不均匀，LLM 的能力呈现出一种奇特的**锯齿形**（jagged）特征——在某些领域登峰造极，在相邻领域却可能犯低级错误。Andrej 将这种现象称为**锯齿形智能**（jagged intelligence）。

问题：为什么同一个模型可以在代码重构上表现惊人，却在常识推理上令人啼笑皆非？

Andrej 举了两个经典例子来说明这种锯齿形：

例 1：草莓字母问题（已被部分修复）。问模型“strawberry 里有几个 r”，早期模型会答错。这个问题看似简单，却暴露了模型在细粒度字符计数上的脆弱性。

例 2：洗车问题（当前仍存）。你告诉模型：“我想去洗车，洗车店距离 50 米，我该开车还是走路？”最先进的模型会建议你**走路**，因为“50 米很近”。

Andrej 的原话是：

”How is it possible that state-of-the-art Opus 4.7 will simultaneously refactor a 100,000 line codebase or find zero-day vulnerabilities and yet tells me to walk to this car wash? This is insane.”

⁰视频画面时间区间：00:09:35–00:09:50。



图 6: 锯齿形智能 (jagged intelligence) ——LLM 在可验证领域登峰造极，却在常识问题上犯低级错误。

为什么简单方法不够：有人可能会想，既然模型在代码上这么强，那常识能力应该也会“顺带”提升。但锯齿形智能的存在表明，能力提升并非均匀扩散。原因在于：

1. **RL 训练的信号密度不同：**代码有编译器、测试套件提供即时反馈；常识没有自动裁判。
2. **数据分布的刻意倾斜：**实验室把算力集中在经济价值高的可验证领域，常识类任务被边缘化。
3. **错误模式的非线性：**模型不是在“从弱到强”平滑进化，而是在特定**电路** (circuits) 上被强化，在其他区域保持原始状态。

收获：锯齿形智能提醒我们，不能因为对模型在某领域的惊艳表现而盲目信任其在所有领域的能力。**你需要了解模型在哪些电路上被训练过**，并据此调整使用策略。

3.3 数据分布的暴政：实验室在训练什么，你就得到什么

锯齿形智能的根源不仅在于可验证性本身，还在于**数据分布** (data distribution) ——即实验室选择把哪些任务放入训练混合 (training mix) 中。

⁰ 视频画面时间区间：00:10:25–00:10:40。

分布外的挣扎

你处于实验室 RL 电路内则飞速进步，处于**分布外**（out-of-distribution, OOD）则举步维艰。Andrej 的原话是：“If you’re in the circuits that were part of the RL, you fly. And if you’re in the circuits that are out of the data distribution, you’re going to struggle.”

这意味着：

- 模型不是“通用智能”的平滑逼近，而是**数据驱动的统计模拟电路**的集合。
- 你在使用模型前，必须先探索它——它“给了你一个没有说明书的东西（something that has no manual）”。
- 你的应用场景如果恰好落在实验室重点训练的数据分布内，你会获得超预期的效果；反之，则可能事倍功半。

核心思想：数据分布不是中性的技术参数，而是**带有价值取向的选择**。实验室倾向于把资源投入到经济价值最高的领域——代码、数学、科学推理——因为这些领域的可验证性使 RL 可以高效展开，产出可量化的能力跃升。

Andrej 指出：“There’s probably lots of verifiable environments they could think about that happen not to make it into the mix because they’re just not that useful to have the capability around.”

如何工作：对于开发者而言，这意味着你需要做两件事：

1. **探测：**在投入大量工程资源前，先用小规模实验测试模型在你的具体场景上的表现。
2. **诊断：**如果表现不佳，判断是因为任务本身不可验证，还是因为你的场景不在当前数据分布内。

收获：不要盲目抱怨“模型不够聪明”。更准确的诊断是：**你的问题是否在它的 RL 电路覆盖范围内？**

3.4 国际象棋的意外跃升：GPT-3.5 到 GPT-4 的启示

Andrej 讲了一个极具启发性的案例：从 GPT-3.5 到 GPT-4，国际象棋能力出现了惊人的跃升。很多人以为这是模型整体能力自然提升的结果，但真相更加微妙。

能力跃升的真相

国际象棋能力的跃升，**主要不是因为模型变“强了”，而是因为数据分布被刻意调整了。**

Andrej 透露：“This is public information... a huge amount of data of chess made it into the pre-training set... someone at OpenAI decided to add this data and now you have a capability that just peaked a lot more.”

问题：为什么这个案例如此重要？

因为它揭示了一个反直觉的事实：**LLM 的能力边界在很大程度上是可塑的、人为的**。同一个基础架构，只要改变训练数据的混合比例，就能在某些领域制造出“能力跃升”的假象。

为什么简单方法不够：如果误以为所有能力提升都来自模型架构或规模的进步，你就会：

- 对未来进展做出错误的指数级 extrapolation；

- 忽视数据工程在 AI 能力塑造中的核心作用；
- 低估在自己领域内构建专用 RL 环境的价值。

如何工作：对于创业者和工程师，这个案例的启示是：

1. 观察到的能力跃升可能有**多种解释**——不要自动归因于“模型变聪明了”。
2. 数据分布是**杠杆点**：在正确的领域注入高质量数据，可以用相对小的成本获得显著的能力提升。
3. 实验室的选择塑造了我们对“AI 能做什么”的集体认知，但这只是**当前分布下的快照**。

下一章将转向 Karpathy 对氛围编程与智能体工程的完整阐述，探讨人类在 AI 协作中的角色重新定义，以及他对智能本质的哲学反思。



图 7: 国际象棋能力的“意外跃升”——GPT-3.5 到 GPT-4 的进步背后，可能是数据分布的刻意调整。

收获：我们在一定程度上受实验室支配 (slightly at the mercy of whatever the labs are doing)。理解这一点，才能在 AI 能力评估中保持清醒，避免被演示效果过度影响判断。

3.5 给创始人的建议：在可验证领域建立 RL 环境

时间范围：13:39 – 15:46 (原 Section 4 内容合并)

基于可验证性框架，Andrej 对在场创始人提出了具体建议。

可验证性让你可以拉“杠杆”⁰

可验证性的最大价值在于：它让你能够**自建 RL 环境**，用自有数据做**微调** (fine-tuning)，从而拉“杠杆” (pull a lever) 获得专属能力。

Andrej 的原话：“Verifiability makes something tractable in the current paradigm because you can throw a huge amount of RL at it... that remains true even if the labs are not focusing on it directly.”

⁰视频画面时间区间：00:13:30–00:13:45。

问题：如果实验室已经在数学和代码上”逃逸速度”了，创业者还能在哪里建立壁垒？

核心思想：可验证性是一个**通用杠杆**，不依赖于实验室是否关注你的领域。只要你的问题域满足以下条件，你就可以自建 RL 管道：

1. 输出可以被自动验证（有明确的正确/错误信号）；
2. 你能构建或收集大量多样化的训练数据；
3. 你有足够的场景来生成 RL 环境（即模型可以尝试-失败-学习的闭环）。

如何工作：Andrej 建议的具体路径：

```
1 # 1. 识别你领域中的可验证信号
2 verification_signals = {
3     "代码": "编译通过□+□测试套件",
4     "设计": "A/B测试转化率", # 较弱但可用
5     "客服": "用户满意度评分", # 需要设计
6     "法律文档": "专家审核一致性" # 半自动
7 }
8
9 # 2. 构建自有RL环境
10 # 关键：奖励信号必须自动化、可规模化
11 # 如果依赖人工标注，RL的效率会大幅下降
12
13 # 3. 使用微调框架"拉杠杆"
14 # 预训练模型 + 领域RL环境 -> 领域专用能力
```

Listing 9: 可验证领域的创业 RL 策略

Andrej 还暗示了一个尚未被充分开发的领域（他故意没有明说），但强调：“There is one domain that I think is very... there are some examples of this.”

收获：不要在实验室已经统治的领域正面竞争。寻找**高价值、可验证、但尚未被大规模 RL 覆盖**的垂直领域，建立你自己的数据-训练飞轮。

3.6 一切终将可自动化——只是难易之分

访谈者追问：“还有什么领域看起来只能从远处自动化？”Andrej 的回答简洁而有力：

”I do think that ultimately almost everything can be made verifiable to some extent. Some things easier than others... everything is automatable.”

问题：如果一切终将自动化，那”安全领域”的边界在哪里？

核心思想：Andrej 认为，**可验证性是一个连续光谱，而非二元开关**。即使对于看似主观的领域（如写作），也可以通过设计间接的验证机制来推进自动化：

$$\text{可自动化程度} \propto \frac{\text{验证信号的自动化密度}}{\text{任务的主观模糊度}}$$

其中：

- **验证信号的自动化密度：**单位时间内能获得的、无需人工干预的正确性反馈数量；
- **任务的主观模糊度：**不同人类专家对”正确答案”的一致程度。

如何工作：对于写作这类低可验证性领域，Andrej 提出了一个创造性的解决方案：

”Even for things like writing, you can imagine having a council of LLM judges and probably get something reasonable out of this kind of approach.”

这意味着：**当单一裁判不可行时，可以用多智能体评审机制来构造人工验证信号。**

收获：

1. 不要过早宣布某个领域”AI 做不了”——问题可能只是你还没有找到构造验证信号的方法。
2. 自动化的速度差异巨大：代码可能是”光速”，创意写作可能是”步行速度”，但方向是一致的。
3. 对于创业者，**找到构造验证信号的方法本身就是巨大的商业机会。**

3.7 本章小结

本章围绕**可验证性**（verifiability）这一核心框架，解释了 LLM 能力分布的深层规律：

1. **可验证性决定自动化速度：**传统计算机自动化”可编码”的，LLM 自动化”可验证”的。强化学习依赖自动化的奖励信号，因此可验证领域的能力进步最快。
2. **锯齿形智能是常态：**LLM 不是均匀变强的。它能在代码重构和零日漏洞发现上登峰造极，却可能在”50 米外洗车该开车还是走路”这类常识问题上犯错。这种**锯齿形智能**（jagged intelligence）源于 RL 训练的不均匀覆盖。
3. **数据分布塑造能力边界：**我们观察到的能力跃升（如 GPT-3.5 到 GPT-4 的国际象棋进步），往往不只是模型变强了，更是**训练数据分布被人为调整了**。我们一定程度上受实验室选择支配。
4. **创始人应自建 RL 环境：**可验证性是一个可以拉”杠杆的”技术。在实验室尚未覆盖的高价值垂直领域，用自有数据构建 RL 环境并做微调，是建立壁垒的有效路径。
5. **一切终将自动化，只是快慢之别：**可验证性是一个连续光谱。即使主观领域，也可以通过多智能体评审等机制构造间接验证信号。关键在于**验证信号的自动化密度**。

下一章将转向 Karpathy 对氛围编程与智能体工程的完整阐述，探讨人类在 AI 协作中的角色重新定义，以及他对智能本质的哲学反思。



图 8: 处于 RL” 电路” 内则飞速进步，处于分布外则举步维艰。

4 从氛围编程到智能体工程

时间范围：15:46–25:17

本章是全篇最长、最丰富的章节，Andrej Karpathy 在此完整阐述了他对 AI 时代软件开发范式的核心观点。从”氛围编程”（vibe coding）到”智能体工程”（agentic engineering），从人类角色的重新定义到对智能本质的哲学反思，本章涵盖了他对未来编程、招聘、教育乃至”何为智能”的深层思考。理解本章，是理解整篇访谈精神内核的关键。

4.1 Vibe Coding：抬高地板，让每个人都能写软件

问题：当 AI 能自动生成代码时，编程的门槛会发生什么变化？

Andrej 首先回顾了他自己提出的术语——氛围编程（vibe coding）。这个词描述的是一种开发方式：开发者完全信任 AI 代理生成代码，不再逐行手动修正，仅凭”氛围”和直觉驱动开发。在 2024 年 12 月模型能力质变之后，他发现”代码块直接就能跑通”，然后”不断要更多功能，它也一直能跑通”，以至于”记不清上次手动修正是什么时候”。

这种体验的核心在于，AI 大幅降低了编程的准入门槛。以前需要多年训练才能掌握的语法、框架、调试技巧，现在被一个能听懂自然语言的代理所替代。不会写代码的人，也能通过描述需求让 AI 生成可用的软件。

核心概念：地板与天花板

Vibe Coding = 抬高地板：它让原本不会编程的人也能创造软件，极大扩展了软件生产者的基数。

Agentic Engineering = 保住质量天花板：在专业软件领域，不能因为 AI 辅助就降低安全、可靠性和质量底线。

⁰视频画面时间区间：00:14:00–00:14:15。

为什么简单方法不够：但 Andrej 立刻指出，如果仅仅停留在 vibe coding 的层面，是有风险的。个人 side project 可以靠“氛围”驱动，但专业软件——尤其是那些处理用户数据、涉及支付、需要长期维护的系统——不能仅靠“感觉”。

核心思想：Vibe coding 的真正价值在于**抬高地板 (raise the floor)**。它让每个人都能参与软件创造，这是前所未有的民主化力量。但它本身并不解决专业软件的质量问题。

收获：对于学习者而言，vibe coding 是一把双刃剑。它降低了入门门槛，但也可能让新手误以为“编程就是和 AI 聊天”。真正重要的，是在享受低门槛创造的同时，理解何时需要从 vibe coding 升级到更严肃的工程实践。

4.2 Agentic Engineering：保住天花板，专业软件的质量底线

问题：如果 vibe coding 让每个人都能写代码，专业工程师的价值在哪里？

Andrej 提出了**智能体工程 (agentic engineering)**，首次出现：英文 agentic engineering，指在氛围编程之上更严肃的工程学科，强调在利用 AI 加速的同时保住专业软件的质量底线和安全标准) 这一概念，作为 vibe coding 的必要补充。

他将 Agent 描述为“spiky entities”——它们是尖刺状的、有点脆弱 (fable)、带有随机性 (stochastic) 但极其强大的实体。问题在于：如何协调这些强大但不可靠的代理，在加速开发的同时不牺牲质量底线？

核心思想：Agentic engineering 是一门真正的工程学科。它回答的问题是：“你仍然要对自己的软件负责，和以前一样。但你能不能走得更快？答案是能，但如何正确做到？”

这意味着：

- 不允许因为 vibe coding 而引入安全漏洞
- 不允许降低代码审查和测试标准
- 需要建立与 Agent 协作的工程流程

收获：对于专业开发者，Agentic engineering 意味着重新设计工作流——不是替代人类工程师，而是让工程师成为 Agent 的“导演”，负责审美、判断和高层设计，将执行细节交给 Agent 填充。



图 9: Vibe Coding 抬高地板，让每个人都能写软件；Agentic Engineering 保住并推高质量天花板。

4.3 10 倍工程师？不，是 100 倍

问题：AI 辅助下，顶尖开发者的效率上限在哪里？

硅谷长期以来有“10 倍工程师”（10x engineer，首次出现：英文 10x engineer，传统硅谷概念，指效率远超普通工程师的顶尖开发者；Andrej 认为 AI 时代这一倍数被大幅放大）的说法，指那些效率远超普通工程师的顶尖开发者。Andrej 认为，在 Agentic engineering 的范式下，这个倍数被大幅放大了。

核心论点：“10 倍不是你能获得的速度提升。”那些真正精通 Agentic engineering 的人，其峰值效率远超 10 倍。

为什么：因为 AI 代理处理的是执行层面的工作——写代码、查文档、跑测试、修 bug——而人类工程师专注于架构设计、需求理解和质量把控。这种分工让工程师的“杠杆”被极大延长。一个优秀的工程师加上一个强大的 Agent，其产出可能相当于过去一个中等规模团队。

收获：这改变了我们对“优秀工程师”的定义。未来的核心竞争力不是手写代码的速度，而是：

1. 设计清晰规格的能力
2. 判断 Agent 输出质量的能力
3. 协调多个 Agent 完成复杂项目的能力

4.4 AI 原生 vs 平庸使用者：工具投资的差距

问题：同样使用 AI 编码工具，为什么有人效果显著，有人却只是把它当搜索引擎？

Andrej 借用了 Sam Altman 的一个观察：不同代际的人使用 ChatGPT 的方式截然不同——30 多岁的人把它当 Google 搜索替代品，而青少年则把它当作通往互联网的入口。在编程领域，同样的分化正在发生。

AI 原生开发者的标志

AI-native（AI 原生的，首次出现：英文 AI-native，深度掌握并最大化利用 AI 工具特性的开发者和 workflows，与仅将 AI 当作搜索替代品的用法相对）开发者的核心特征：

- **最大化利用工具特性：**不只是用 AI 生成代码片段，而是利用 Agent 的完整能力——规划、执行、调试、测试
- **深度投资个人工作流：**像过去投资 Vim/VS Code 配置一样，投资 Claude Code、Codex 等 Agent 工具的配置和定制
- **持续迭代工具链：**不断尝试新工具、新工作流，找到最适合自己的组合

核心思想：平庸使用者把 AI 当作“更好的自动补全”，而 AI 原生开发者把 AI 当作“协作伙伴”。这种认知差距导致了效率的指数级分化。

收获：成为 AI 原生开发者不是天赋，而是选择。它要求你像对待核心技能一样对待工具投资——花时间学习 Agent 工具的高级特性，定制自己的工作流，而不是满足于最基础的使用方式。

4.5 招聘范式转变：从解谜题到做大项目

问题：当 AI 能轻松解决算法谜题时，如何评估工程师的真实能力？

⁰视频画面时间区间：00:15:55–00:16:10。

Andrej 指出了正在发生但大多数人尚未意识到的转变：传统的招聘流程已经过时了。

还在用算法谜题招聘？

如果你还在给候选人出算法谜题 (puzzles)，你仍然活在旧范式里。在 Agentic engineering 时代，这种评估方式完全无法衡量一个人与 AI 协作构建复杂系统的能力。

新的招聘范式：Andrej 建议的评估方式是”给一个大项目”：

1. 让候选人用 Agent 构建一个 Twitter 克隆
2. 要求将其做得足够好、足够安全
3. 让 Agent 模拟用户活动
4. 然后用多个 Agent（如”10 个 Codex 5.4x”）进行红队测试（red teaming，首次出现：英文 red teaming，用对抗性方式测试系统安全性和鲁棒性的方法，Andrej 建议用于评估 Agent 构建的应用），尝试攻破它
5. 候选人需要确保系统不被攻破

核心思想：这种测试评估的是候选人在真实工程场景中的能力——系统设计、安全思维、与 Agent 协作、以及质量把控。这些才是 Agentic engineering 时代真正重要的技能。

收获：对于求职者和招聘者，这意味着需要重新思考”能力”的定义。未来的工程师面试可能更像一场”黑客马拉松”——在限定时间内，用 Agent 构建、部署并加固一个真实应用。



图 10: AI 时代的招聘范式转变——从算法谜题到真实项目 + 红队测试。

4.6 Agent 是”实习生”：有强大记忆力，但缺乏常识

问题：当 Agent 承担更多工作时，人类应该扮演什么角色？

⁰视频画面时间区间：00:19:25–00:19:40。

Andrej 用了一个精准的比喻：当前的 Agent 就像**实习生** (intern, 首次出现：英文 intern, Andrej 对当前 Agent 能力的比喻：有强大记忆力和执行力，但缺乏常识和判断力，需要人类监督)。它们有强大的记忆力和执行力，但缺乏常识和判断力，需要人类的监督和指导。

MenuGen 的邮件地址关联 bug

Andrej 分享了一个真实案例。在 MenuGen 应用中：

- 用户用 Google 账号登录
- 用户用 Stripe 购买积分
- 两个系统都有邮箱地址

Agent 的“解决方案”是：用 Stripe 的邮箱地址去匹配 Google 的邮箱地址来关联用户。问题是——用户完全可能在两个服务中使用不同的邮箱！这导致购买记录无法正确关联到用户账户。

Andrej 的评价是：“这太奇怪了。为什么会用邮箱地址来做跨服务关联？邮箱可以是任意的。”这是典型的“有代码能力但无常识”的表现。

核心思想：Agent 会在你意想不到的地方犯错。它们擅长执行明确的指令，但在需要常识判断的场景（“用户可能在不同服务用不同邮箱”）上仍然薄弱。

收获：人类必须负责规格 (spec) 和计划的制定。Agent 是执行者，人类是导演。你不能把设计思维和常识判断外包给 Agent。

4.7 品味与判断：人类仍然负责的核心能力

问题：在 AI 能生成代码、写文档甚至设计架构的时代，人类还有什么不可替代的价值？

Andrej 的答案是：**品味** (taste) 和**判断力** (judgment, 首次出现：英文 taste / judgment, 人类在 AI 协作中仍然不可替代的核心能力，包括美学判断、架构设计和需求理解)。

核心论点：即使 Agent 能生成代码，人类仍然必须负责：

- **美学判断：**界面是否好看、交互是否流畅
- **架构设计：**系统的整体结构是否合理
- **需求理解：**是否在建正确的东西
- **质量把控：**代码是否可维护、是否安全

当被问及“品味和判断力是否会随着时间变得不那么重要”时，Andrej 的回答很诚实：他希望会改善，但目前没有——因为“审美成本或奖励”不在强化学习的训练目标中。实验室还没有为此优化。

收获：这意味着，未来的工程师教育应该更加重视“品味”的培养——不是教具体的 API 用法，而是教什么是好的设计、什么是清晰的架构、什么是值得建的东西。

4.8 规格设计 (Spec) 的艺术：从 Plan Mode 到文档驱动

问题：如果人类负责规格而 Agent 负责执行，那么“规格”应该长什么样？

Andrej 对当前 AI 编码工具中的”规划模式” (plan mode, 首次出现: 英文 plan mode, AI 编码工具中的功能, 让 Agent 在执行前先制定详细计划; Andrej 认为这只是一般性需求的一部分) 持保留态度。他认为 plan mode 有用, 但只是更一般性需求的一部分。

核心思想: 真正重要的是与 Agent 协作设计一份**详细的规格说明** (spec, 首次出现: 英文 spec / specification, 详细的设计文档和需求规格, 人类负责制定, Agent 负责按规格实现)。这份规格可能表现为文档 (docs), 人类负责高层分类和监督, Agent 负责填充底层细节。

工作模式:

1. 人类定义高层架构和核心约束 (如”所有用户必须用唯一用户 ID 关联, 不能用邮箱”)
2. Agent 根据规格生成实现代码
3. 人类审查输出, 确保符合设计意图
4. 迭代 refine, 直到满足质量标准

收获: 规格设计是 Agentic engineering 的核心技能。好的规格要足够详细以避免 Agent”自由发挥”导致常识性错误, 又要足够高层以给 Agent 留下执行空间。这是一门需要练习的艺术。

4.9 细节可以外包, 原理必须掌握

问题: 当 Agent 能记住所有 API 细节时, 人类还需要知道什么?

Andrej 用一个具体的例子说明了”细节外包”与”原理掌握”的界限。

你可以忘记 API 细节, 但必须理解视图 vs 拷贝

Andrej 坦言: ”我已经忘了 keepdims 还是 keep_dim, 忘了是 dim 还是 axis, 忘了 reshape 还是 permute 还是 transpose。我不记得这些东西了, 因为不需要。”

但他强调, 你**必须**理解底层原理:

- 张量 (tensor, 首次出现: 英文 tensor, 深度学习中的核心数据结构概念) 底层有视图 (view, 首次出现: 英文 view, 张量的视图, 与存储共享数据) 和存储 (storage, 首次出现: 英文 storage, 张量数据的实际存储)
- 你可以操作同一存储的不同视图, 也可以创建独立存储
- 独立存储效率更低 (需要额外内存拷贝)

”你必须理解这些东西在做什么, 以及基本原理, 这样才不会不必要地拷贝内存。”

核心思想: API 细节是”事实知识”, 可以被外包给 Agent; 但底层原理是”概念理解”, 必须掌握。因为原理决定了你能否做出正确的设计决策。

收获: 这重新定义了”学习编程”的含义。未来不是背诵 API, 而是理解:

- 数据结构在内存中如何表示
- 算法的时间和空间复杂度权衡
- 系统的架构模式和设计原则



图 11: 品味与判断力——人类在 AI 协作中不可替代的核心能力。

4.10 为什么模型讨厌简化：RL 电路之外的挣扎

问题：为什么 Agent 能写复杂代码，却难以写出简洁优雅的代码？

Andrej 分享了一个令人沮丧的经历。他在做 microGPT 项目时，试图让 LLM 将代码简化到最简形式——“不断要求简化、再简化”。

让模型简化代码像“拔牙”

Andrej 的描述非常生动：“模型讨厌这个。它们做不到。你感觉自己在 RL 电路之外。感觉像是在拔牙。不像光速。”

这背后的原因是：

- 模型的训练数据（代码库）中，复杂代码远多于简洁代码
- 强化学习环境奖励“能工作”的代码，而非“优雅”的代码
- 简化代码是一种审美判断，而如 6.2 节所述，审美不在 RL 目标中

核心思想：当你要求模型做“简化”时，你处于它的 RL 电路之外。模型在“写能工作的代码”这个电路里飞快地运转，但在“写优雅的代码”这个电路里举步维艰。

收获：这再次印证了人类品味的不可替代性。Agent 可以帮你快速搭建原型，但代码的精炼、架构的优雅、抽象的恰当——这些仍然需要人类工程师的审美判断和手动调整。

4.11 我们不是在建动物，我们在召唤幽灵

问题：我们到底在构建什么？是具有内在智能的“生命”，还是某种更奇怪的东西？

从 23:36 开始，Andrej 进入了整篇访谈最哲学性的段落——**动物 vs 幽灵**（animals vs ghosts，首次出现：英文 animals vs ghosts，Andrej 的哲学比喻：我们不是在建具有内在动机和进化的动物智能，而是在召唤由数据和奖励函数塑造的幽灵）。

⁰视频画面时间区间：00:21:05–00:21:20。

对 Agent 大喊不会让它工作得更好

Andrej 的核心观点：“如果你对它大喊，它不会工作得更好或更差。这没有任何影响。”这不是动物智能。动物会对鼓励、惩罚、情绪做出反应。Agent 不会——因为它不是通过进化产生的生命体，没有内在动机，没有情感回路。

核心思想：我们不是在“建造”动物（通过数百万年进化，具有内在动机和适应性的生命），我们是在“召唤”幽灵——由海量数据和奖励函数塑造的、能力强大但本质陌生的智能形态。

为什么这个比喻重要：如果你把 Agent 当作动物（或人），你会错误地期待：

- 鼓励会让它更努力（不会）
- 批评会让它改正（不会）
- 给它更多时间会让它思考更深（不会）

Agent 的行为完全由其内部的统计电路决定，而非情绪、动机或自我意识。



图 12: 动物 vs 幽灵——我们不是在建具有内在动机的动物智能，而是在召唤由数据和奖励函数塑造的幽灵。

4.12 统计模拟电路：预训练 + RL 的叠加

问题：如果 Agent 是“幽灵”，它的内部结构是什么样的？

Andrej 用**统计模拟电路**（statistical simulation circuits，首次出现：英文 statistical simulation circuits，Andrej 对 LLM 内部工作机制的描述：预训练提供统计基础，RL 在之上叠加特定能力）来描述 LLM 的内部工作机制。

核心模型：

$$\text{模型行为} = \underbrace{\text{预训练统计基础}}_{\text{substrate}} + \underbrace{\text{RL 叠加能力}}_{\text{bolting on top}} \quad (1)$$

⁰视频画面时间区间：00:23:30–00:23:45。

其中：

- **预训练 (pre-training)**：在海量互联网数据上学习语言的统计规律，构成模型的”基底”
- **RL (Reinforcement Learning, 强化学习, 首次出现：英文 RL, 通过奖励信号训练模型的机器学习方法)**：在预训练基础上，通过验证奖励信号叠加特定能力（如代码生成、数学推理）

核心思想：预训练提供的是统计性的语言理解，RL 在此基础上”螺栓固定” (bolting on top) 特定能力。这解释了为什么模型在某些领域（代码、数学）极强，而在其他领域（常识推理）薄弱——因为 RL 只在可验证的领域有效地”螺栓固定”了能力。

收获：理解这个结构有助于预测模型的行为边界。如果一个问题处于”RL 电路”内，模型会表现出色；如果在”电路外”，则表现可能出人意料地差。

4.13 没有内在动机，没有好奇心，没有恐惧

问题：”幽灵”与”动物”的根本区别是什么？

Andrej 明确指出，当前 AI 系统不具备以下通过进化产生的生物特性：

- **内在动机** (intrinsic motivation, 首次出现：英文 intrinsic motivation, 生物进化产生的内在驱动力，当前 AI 系统不具备)：Agent 不会”想要”做好，它只是在执行统计预测
- **好奇心**：Agent 不会主动探索未知，只会响应提示
- **恐惧**：Agent 不会担心失败，不会从错误中学习（除非通过 RL 训练）
- **求生欲**：Agent 没有自我保护的冲动

核心思想：这些缺失不是缺陷，而是本质差异。动物智能是在进化压力下形成的适应性系统，具有平滑的能力曲线和内在的驱动力。幽灵智能是在数据分布和奖励函数下形成的统计系统，具有**锯齿形智能** (jagged intelligence, 首次出现：英文 jagged intelligence, LLM 能力分布不均匀的现象)——在某些维度上超人类，在另一些维度上近乎幼稚。

收获：不要用人性化的框架去理解 Agent。它不会”努力”、不会”懒惰”、不会”生气”。它只是一个极其复杂的统计电路，在特定输入下产生特定输出。这种认知虽然”去魅”，但能让你更准确地预测和控制它的行为。

4.14 这种认知如何改变使用策略

问题：如果 Agent 是”幽灵”而非”动物”，我们应该如何调整使用它的方式？

Andrej 承认，这个框架更多是一种”哲学思考” (philosophizing)，他不确定是否有”五个显而易见的结论”。但他认为，这种认知至少带来以下改变：

保持怀疑，持续探索

Andrej 的最终建议：“保持怀疑，持续探索，没有操作手册。”

具体策略：

1. **不要情绪化交互**：对 Agent 大喊、恳求、鼓励都没有用。专注于清晰的规格和约束。
2. **识别你的电路**：探索模型在哪些任务上表现好（在 RL 电路内），哪些任务上表现差（在电路外）。
3. **持续实验**：模型能力分布没有手册，只有通过实践才能了解边界。
4. **怀疑默认值**：Agent 的默认输出可能臃肿、脆弱，需要人类品味来 refine。

核心思想：使用 Agent 的正确心态不是“雇佣一个聪明的助手”，而是“操作一台强大但陌生的机器”。你需要了解它的特性、边界和怪癖，而不是假设它像人类一样“理解”你的意图。

收获：这种认知框架虽然听起来冷酷，但它是有效使用 Agent 的前提。接受 Agent 的“非人性”，反而能让你更好地与它协作——因为你会更仔细地设计规格、更严格地验证输出、更清醒地认识边界。

4.15 本章小结

本章是 Andrej Karpathy 对 AI 时代软件开发范式的完整阐述，核心论点可以总结为三个层次：

第一层：两种编程范式

- **Vibe Coding** 抬高地板，让每个人都能创造软件
- **Agentic Engineering** 保住天花板，确保专业软件的质量底线
- 两者不是对立，而是互补——前者扩展可能性，后者确保可靠性

第二层：人的角色重新定义

- Agent 是“实习生”——有执行力，缺常识
- 人类负责品味、判断和规格设计
- API 细节可以外包，底层原理必须掌握
- 招聘从“解谜题”转向“做大项目 + 红队测试”

第三层：智能的本质反思

- 我们不是在建造动物，而是在召唤幽灵
- Agent 是统计模拟电路——预训练基底 + RL 叠加
- 没有内在动机、好奇心或恐惧
- 对 Agent 大喊不会让它工作得更好

关键金句：

“10 倍不是你能获得的速度提升。”——Andrej Karpathy

”你可以外包思考，但不能外包理解。”——引自 Andrej 引用的一条推文

行动启示：

1. 投资你的 Agent 工具链，成为 AI 原生开发者
2. 练习规格设计——这是与 Agent 协作的核心技能
3. 保持对底层原理的理解，即使 API 细节可以忘记
4. 对 Agent 保持健康的怀疑，持续探索其能力边界
5. 培养品味和判断力——这是人类不可替代的最后堡垒

下一章将探讨当 Agent 真正拥有行动能力时，我们的数字基础设施需要怎样被重写，以及在智能变得廉价的时代，人类教育的核心目标应当是什么。

5 智能体无处不在：基础设施与教育

时间范围：25:17 – 29:49

Andrej Karpathy 在访谈的最后阶段，将话题从个人开发体验与哲学反思，转向了更具前瞻性的两个议题：一是当 Agent 真正拥有行动能力时，我们的数字基础设施需要怎样被重写；二是在智能变得廉价的时代，人类教育的核心目标应当是什么。这两个问题共同指向一个判断——人类不会消失，但人类必须重新定义自己的角色。



图 13: Agent 原生基础设施的呼吁——一切为人类写的文档都是 Agent 的障碍。

⁰视频画面时间区间：00:25:10–00:25:25。

5.1 Agent 原生基础设施：一切为人类写的文档都是负担

5.1.1 问题：为什么今天的文档让 Agent 开发者抓狂

当我们使用任何框架、库或云服务时，第一步通常是阅读官方文档。文档会告诉你：“去这个 URL 注册”、“在设置菜单里勾选这个选项”、“复制这段配置到你的文件中”。这些指令对人类来说是自然的，但对 Agent 来说却是灾难。

Andrej 直言这是他最大的“怨念” (pet peeve)：

“为什么人们还在告诉我该做什么？我不想做任何事。我应该复制粘贴什么东西给我的 Agent？”

这一抱怨揭示了一个深层矛盾：当前整个数字世界的基础设施——从 API 文档到云服务控制台，从 DNS 配置界面到支付网关设置——都是为人类设计的。人类有视觉、有直觉、能点击下拉菜单、能理解示意图。但 Agent 没有这些能力，它只能读取文本、调用 API、执行命令。当 Agent 试图按照为人类编写的文档去操作时，每一步都需要人类介入翻译，自动化链条就此断裂。

5.1.2 为什么简单方法不够：给 Agent 看人类文档是“二次翻译”

有人可能会想：既然 Agent 能读文档，那让它直接读不就行了？问题在于，为人类编写的文档本质上是“描述性”的，而非“可执行”的。

以部署一个 Web 应用为例，人类文档可能会说：

1. 前往 Vercel 官网并登录；
2. 点击 “New Project” 按钮；
3. 选择你的 GitHub 仓库；
4. 在环境变量页面添加你的 API 密钥；
5. 配置自定义域名，前往 DNS 提供商添加 CNAME 记录。

对 Agent 而言，每一步都需要解析自然语言、识别 UI 元素、模拟点击操作。这相当于让 Agent 做“二次翻译”：先把人类语言翻译成意图，再把意图翻译成机器操作。这个过程脆弱、易错，且每次界面改版都会破坏自动化。

5.1.3 核心理念：Agent 原生 (Agent-Native) 的基础设施

别再告诉我该做什么，告诉我该复制粘贴什么给我的 Agent

Agent 原生基础设施的核心原则：所有接口、文档和配置都应当以 Agent 为第一用户设计。这意味着提供机器可读的规格说明、一键可执行的命令、以及结构化的数据表示，而不是面向人类的图文教程。

Andrej 所呼吁的“Agent 原生” (Agent-Native)，是指从设计之初就以 AI Agent 为主要交互对象的基础设施范式。具体来说，它应当具备以下特征：

- **机器可读优先**：文档不是给人看的 HTML 页面，而是给 Agent 解析的结构化文本（如 JSON Schema、OpenAPI 规格、或可直接粘贴到提示词中的配置块）。

- **可执行性**：提供“复制粘贴即可运行”的代码片段或命令，而非需要人类判断和点击的 GUI 操作说明。
- **传感器-执行器接口**：将服务分解为 Agent 可以直接感知的输入（传感器）和可以直接触发的动作（执行器），消除中间的人工翻译层。

5.1.4 如何工作：从人类文档到 Agent 接口

想象一个 Agent 原生的云服务部署流程。人类开发者只需要对 Agent 说：

```

1 部署我的MenuGen应用到生产环境。
2 要求：
3 - 使用Vercel作为托管平台
4 - 配置Stripe支付网关
5 - 设置Google OAuth登录
6 - 绑定自定义域名 menu.example.com
7 - 启用HTTPS
8
9 以下是各服务的Agent配置块：
10
11 [VERCEL_CONFIG]
12 project_name: menugen-prod
13 framework: nextjs
14 env_vars: {STRIPE_KEY: ..., GOOGLE_CLIENT_ID: ...}
15
16 [STRIPE_CONFIG]
17 webhook_endpoint: https://menu.example.com/api/webhook
18 products: [credits_pack_10, credits_pack_50]
19
20 [DNS_CONFIG]
21 domain: menu.example.com
22 target: cname.vercel-dns.com

```

Listing 10: Agent 原生部署指令示例

Agent 接收到上述指令后，可以直接调用各服务的 API 完成全部配置，无需人类点击任何按钮。每个配置块都是机器可解析的结构化数据，而非需要人类理解的自然语言描述。

5.1.5 收获：基础设施范式的又一次迁移

Agent 原生基础设施的变革，其意义不亚于从命令行到图形界面、或从本地服务器到云计算的迁移。它意味着：

- **自动化从代码层扩展到运维层**：不仅写代码可以自动化，部署、配置、监控、扩容都可以由 Agent 闭环完成。
- **人类从操作者变为审批者**：人类只需要确认 Agent 的执行计划，而不需要亲自执行每一步。
- **新的创业机会**：第一批提供 Agent 原生接口的云服务和开发工具，将获得巨大的开发者粘性。



图 14: ”别再告诉我该做什么，告诉我该复制粘贴什么给我的 Agent。”

5.2 传感器与执行器：分解世界的 Agent 视角

5.2.1 问题：Agent 如何感知和影响现实世界

如果 Agent 要成为数字世界中的自主行动者，它必须能够接收信息（感知）和做出改变（行动）。但今天的软件系统并不是按照这一模型设计的——它们是为人类用户设计的，人类通过眼睛看屏幕、通过手指点击按钮。

5.2.2 为什么简单方法不够：人类中心的设计限制了自动化

传统软件架构通常围绕“用户界面”（UI）展开。数据库有管理后台，云服务有 Web 控制台，支付平台有 Dashboard。这些 UI 对人类友好，但对 Agent 来说是“不透明”的。Agent 无法“看见”一个网页上的按钮，也无法“理解”一个图表所表达的趋势。如果 Agent 想要获取信息或执行操作，它必须依赖专门的 API——而这些 API 往往覆盖不全、文档不清、或者根本不存在。

5.2.3 核心思想：用传感器与执行器重新分解世界

Andrej 借用机器人学的经典框架，提出了一个 Agent 原生的世界观：将一切系统分解为**传感器**（sensors）和**执行器**（actuators）。

- **传感器**：Agent 感知世界的输入端。包括文件系统状态、数据库查询结果、API 响应、日志流、监控指标、甚至网页的原始 HTML。
- **执行器**：Agent 影响世界的输出端。包括写入文件、调用 API、执行命令、发送请求、修改配置。

这一分解方式的美妙之处在于它的普适性。无论是操作本地代码仓库、管理云服务器、还是与第三方服务交互，Agent 都可以通过“读取传感器 → 内部推理 → 触发执行器”的循环来完成任务。

⁰视频画面时间区间：00:25:50–00:26:05。

5.2.4 如何工作：传感器-执行器循环

一个典型的 Agent 工作循环可以表示为：

$$\text{State}_{t+1} = f(\text{State}_t, \text{Sensor}(\text{Env}), \text{Prompt})$$

其中：

- State_t ：Agent 在时刻 t 的内部状态（上下文窗口中的内容）；
- $\text{Sensor}(\text{Env})$ ：Agent 从环境（Env）中通过传感器读取的信息；
- Prompt：人类给 Agent 的任务指令；
- f ：LLM 的推理函数，根据当前状态、感知输入和任务指令，决定下一步行动；
- State_{t+1} ：更新后的状态，包含 Agent 决定触发的执行器动作。

这个循环与强化学习中的“观察-行动-奖励”框架异曲同工，但在这里，Agent 的“奖励”来自于任务是否成功完成，而非显式的数值信号。

5.2.5 收获：一切系统都可以被 Agent 化

传感器-执行器的视角提供了一种系统性的方法，来判断任何现有系统是否“Agent 就绪”：

1. 该系统是否提供机器可读的传感器接口？（Agent 能否获取它需要的信息？）
2. 该系统是否提供机器可调用的执行器接口？（Agent 能否执行它需要的操作？）
3. 传感器和执行器之间的反馈循环是否闭合？（Agent 能否验证它的操作是否成功？）

如果答案都是肯定的，那么这个系统就是 Agent 原生的。否则，它就是需要被“重写”的候选者。



图 15: 传感器与执行器——分解世界的 Agent 视角。

⁰视频画面时间区间：00:26:05–00:26:20。

5.3 从部署地狱到一键部署：MenuGen 的教训

5.3.1 问题：写代码容易，部署代码难

Andrej 以他自己开发的 MenuGen 应用为例，讲述了一个令所有开发者共鸣的痛苦经历。MenuGen 的核心功能——拍摄菜单照片、OCR 识别菜品、生成菜品图片——本身并不复杂，在 Agent 的辅助下，代码编写相对顺利。但真正的噩梦发生在部署阶段。

5.3.2 为什么简单方法不够：现代部署流程充斥着人类操作

当 Andrej 尝试将 MenuGen 部署到 Vercel 时，他遇到了一连串需要人工干预的障碍：

- 需要前往 Vercel 的控制台创建项目；
- 需要将 GitHub 仓库与 Vercel 项目关联；
- 需要前往 Stripe 后台配置支付和 Webhook；
- 需要前往 Google Cloud Console 配置 OAuth 登录；
- 需要前往 DNS 提供商配置域名解析；
- 需要在各个服务的设置页面之间来回跳转，确保配置一致。

Andrej 形容这个过程“极其烦人”（so annoying）。这些操作对人类来说已经够繁琐了，对 Agent 来说几乎是不可能的——因为每一步都涉及不同网站的 GUI 操作，没有统一的 API 可以调用。

5.3.3 核心思想：部署应当是提示词级别的操作

Andrej 提出了一个雄心勃勃的测试标准：

“我应该能够给 LLM 一个提示词——‘构建 MenuGen’——然后我就不需要碰任何东西，它就能部署到互联网上。”

这一标准将部署的复杂度从“多步骤人工操作”降低到了“单条自然语言指令”。实现这一目标需要：

1. **声明式配置**：所有部署参数（域名、环境变量、依赖服务）都以结构化方式提供，Agent 可以直接解析；
2. **服务间 API 互通**：Vercel、Stripe、Google OAuth、DNS 提供商之间提供标准化的 Agent 接口，Agent 可以直接调用而无需人类点击；
3. **闭环验证**：部署完成后，Agent 能够自动验证服务是否正常运行（健康检查、端到端测试）。

5.3.4 如何工作：理想中的 Agent 部署流程

1	构建并部署 MenuGen 应用到生产环境。
2	
3	功能需求：
4	- 用户可以上传餐厅菜单照片
5	- 系统使用 OCR 提取菜品名称

```
6 - 系统使用图像生成器为每道菜生成图片
7 - 用户可以通过Google账号登录
8 - 用户可以通过Stripe购买积分
9
10 基础设施需求：
11 - 托管：Vercel，Next.js框架
12 - 数据库：PostgreSQL
13 - 图像生成：调用外部API
14 - 支付：Stripe，webhook自动配置
15 - 认证：Google OAuth
16 - 域名：menu.example.com
17
18 请自动完成所有配置，并在部署后运行端到端测试验证。
```

Listing 11: MenuGen 的理想 Agent 部署指令

在这个理想流程中，Agent 会：

1. 读取上述规格，理解所有依赖；
2. 调用 Vercel API 创建项目并推送代码；
3. 调用 Stripe API 配置产品和 Webhook；
4. 调用 Google OAuth API 配置客户端；
5. 调用 DNS API 配置域名解析；
6. 运行测试验证上传、OCR、图像生成、支付全流程。

整个过程无需人类点击任何一个网页按钮。

5.3.5 收获：MenuGen 是 Agent 原生基础设施的试金石

MenuGen 的部署经历揭示了一个关键洞察：

人类成为瓶颈

在 Agent 时代，最大的瓶颈不再是代码编写能力，而是人类必须介入的“最后一公里”——知道要建什么、为什么值得建、以及如何指挥 Agent 完成部署。如果基础设施不进化，人类将永远被困在繁琐的配置工作中，无法释放 Agent 的全部潜力。

Andrej 将 MenuGen 的部署体验视为衡量基础设施 Agent 化程度的“试金石”。当有一天，给 Agent 一个提示词就能从零部署 MenuGen 而无需任何人工干预时，我们才可以说 Agent 原生基础设施真正成熟了。



图 16: 从部署地狱到一键部署——MenuGen 的教训。

5.4 未来：我的 Agent 和你的 Agent 对话

5.4.1 问题：当每个人都有 Agent 时，协作方式会怎样改变

如果 Agent 能够自主行动，那么逻辑上的下一步是：Agent 之间能否直接协作？当两个人需要协调会议时间、交换合同条款、或者对接 API 时，是否可以让各自的 Agent 代为沟通？

5.4.2 为什么简单方法不够：人类中介降低了协作效率

今天的协作流程通常是这样的：

1. 人类 A 产生一个需求；
2. 人类 A 将需求写成邮件/消息发给人类 B；
3. 人类 B 阅读、理解、提出修改意见；
4. 双方来回多轮，最终达成一致；
5. 人类 B 手动执行约定的操作。

这个流程的瓶颈在于人类。人类需要睡眠、会遗忘、阅读速度慢、且同一时间只能处理少量对话。当协作涉及多方、多系统时，效率急剧下降。

5.4.3 核心思想：Agent 作为个人和组织的代表

Andrej 展望了一个“Agent 代表”（agent representation）的世界：

“我将拥有我的 Agent，你将拥有你的 Agent。我的 Agent 会和你的 Agent 对话，来协调我们会议的一些细节，或者处理类似的事情。”

⁰视频画面时间区间：00:26:55–00:27:10。

这意味着：

- 每个人都有自己的 Agent，它了解主人的偏好、日程、约束条件；
- 每个组织也有自己的 Agent，它掌握组织的流程、政策、API 接口；
- 当需要协作时，Agent 之间直接对话，以机器速度达成协议；
- 人类只在关键决策点介入，日常协调完全自动化。

5.4.4 如何工作：Agent-to-Agent 协议

实现 Agent 间协作需要标准化的通信协议。一个可能的交互流程如下：

```
1 Agent_A (代表Andrej): 请求与B安排会议，主题：讨论合作。
2 偏好：下周任意工作日上午，时长30分钟，时区PST。
3
4 Agent_B (代表对方): 收到请求。可用时段：
5 - 周二 10:00-10:30 PST
6 - 周四 14:00-14:30 PST
7 建议周二上午，请确认。
8
9 Agent_A: 周二 10:00-10:30 PST 已确认。
10 正在创建日历事件并发送邀请。
11 会议链接已生成：https://meet.example.com/abc123
12
13 Agent_B: 日历事件已接受。提醒已设置。
14 如需修改，请随时告知。
```

Listing 12: Agent 间会议协调的示例对话

在这个场景中，两个 Agent 以结构化的方式交换信息、协商时间、执行操作（创建日历、发送邀请）。整个过程在毫秒级完成，而人类只需要在收到最终确认时扫一眼即可。

5.4.5 收获：从人机交互到机机交互

Agent 间协作的愿景意味着互联网将从“人机交互”的时代进入“机机交互”的时代。这要求：

- **身份与授权**：Agent 需要有可验证的身份，以及代表主人行事的授权机制；
- **协商协议**：Agent 之间需要标准化的协商语言，类似于今天的 API 协议，但更灵活、更自然语言化；
- **信任边界**：人类需要能够设定 Agent 的权限边界，确保 Agent 不会做出超出授权范围的承诺。

Andrej 认为这一趋势“大致上就是事情发展的方向”，虽然具体形态还有待探索，但方向是明确的。



图 17: 未来图景：我的 Agent 会和你的 Agent 对话。

5.5 教育：可以外包思考，不能外包理解

5.5.1 问题：当智能变得廉价时，还有什么值得学

访谈的最后一个问题回到了教育——这也是 Andrej 通过 Eureka Labs 一直在探索的领域。问题是：当 AI 可以替你写代码、做分析、生成内容时，人类还应该深度学习什么？什么能力是 AI 无法替代的？

5.5.2 为什么简单方法不够：完全依赖 AI 会让人失去方向感

Andrej 坦承，他自己也感受到了成为“瓶颈”的压力：

“我感觉自己正在成为瓶颈——仅仅是因为知道我们要构建什么、为什么值得做、以及如何指挥我的 Agent。”

如果你完全依赖 AI 来思考，你可能会得到很多输出，但你会失去判断这些输出是否正确的能力。你会不知道 Agent 是否走偏了，不知道它的方案是否最优，甚至不知道它是否理解了你的真实需求。没有理解，就没有方向；没有方向，Agent 再强大也只是无头苍蝇。

5.5.3 核心思想：理解是最后的护城河

You can outsource your thinking but you can't outsource your understanding

你可以外包思考（让 AI 去计算、去推理、去生成），但不能外包理解（你必须自己懂）。理解是你指挥 Agent 的前提，是你判断 Agent 输出质量的依据，也是你在复杂系统中保持方向感的锚点。

Andrej 引用了一条令他“每隔一天都会想起”的推文：“你可以外包思考，但不能外包理解。”这句话精准地捕捉了 AI 时代教育的新定位。

⁰ 视频画面时间区间：00:27:20–00:27:35。

- **思考**：具体的计算、推理、编码、写作过程——这些可以交给 Agent。
- **理解**：对问题本质的把握、对系统行为的直觉、对权衡取舍的判断——这些必须内化为人类自己的能力。

5.5.4 如何工作：用 AI 增强理解，而非替代理解

Andrej 分享了他自己利用 AI 增强理解的方法——**LLM 知识库** (LLM knowledge base)。他的工作流是：

1. 每当阅读一篇文章，他会让 LLM 从中提取关键信息，自动构建个人 Wiki；
2. 他会在 Wiki 上向自己的 Agent 提问，从不同角度审视同一信息；
3. 他使用**合成数据生成** (synthetic data generation) 技术，基于固定数据生成变体，以加深理解。

```
1 输入：一篇关于Transformer架构的技术文章
2
3 步骤1：提取与结构化
4 - LLM提取核心概念、公式、历史背景
5 - 存入个人Wiki，链接到已有相关知识
6
7 步骤2：多角度提问
8 - ``请用三句话向一个高中生解释注意力机制``
9 - ``这个设计与2017年原版论文相比有什么关键改进？``
10 - ``如果要在内存受限的设备上实现这个，最大的挑战是什么？``
11
12 步骤3：合成数据生成
13 - 基于文章中的核心公式，让LLM生成5个变体问题
14 - 自己尝试解答，再与LLM的答案对比
15 - 通过主动回忆加深理解
```

Listing 13: Andrej 的知识增强工作流

这个工作流的关键在于：Agent 不是替 Andrej“学”，而是帮他以更多维度、更高效率地加工信息。最终的理解仍然发生在 Andrej 的大脑中。

5.5.5 收获：理解力是 AI 时代的元能力

Andrej 的教育观可以总结为三个层次：

1. **细节可以外包**：PyTorch 的 API 参数名、Vercel 的配置步骤、Stripe 的 Webhook 格式——这些具体细节不需要死记硬背，Agent 的记忆力远超人类。
2. **原理必须掌握**：张量 (tensor) 的视图 (view) 与存储 (storage) 之间的关系、神经网络前向传播与反向传播的本质、分布式系统的 CAP 权衡——这些底层原理必须真正理解，否则你无法判断 Agent 的输出是否正确。
3. **品味与判断不可替代**：知道什么是好的设计、什么值得做、如何在多个可行方案中做出选择——这些需要长期积累的理解力和审美判断力，是 Agent 短期内无法替代的。

人类成为瓶颈——知道要建什么、为什么值得建、如何指挥 Agent

在 Agent 时代，人类的价值不再是“做”，而是“知道”。知道要构建什么、知道为什么值得构建、知道如何指挥 Agent 去构建。这三项能力都根植于理解，而非记忆或计算。如果你放弃了理解的追求，你就放弃了自己在 AI 时代的核心角色。

Andrej 最后强调，LLM 本身并不擅长“理解”——它们擅长的是基于统计模式生成合理的输出。真正的理解，仍然是人类独有的领地。这也是为什么，尽管 Agent 可以做越来越多的事情，人类的教育目标不应该退化，而应该升级：从培养“会做题的人”升级为培养“有理解力的人”。



图 18: “你可以外包思考，但不能外包理解”——智能时代教育的核心原则。

5.6 本章小结

本章探讨了 Agent 从“聊天工具”进化为“行动主体”后，对基础设施和教育两个领域的深远影响。

基础设施层面，Andrej 提出了“Agent 原生”的变革方向：当前所有为人类设计的文档、界面和配置流程，对 Agent 而言都是负担。未来的基础设施应当以 Agent 为第一用户，提供机器可读的接口、一键可执行的配置、以及传感器-执行器分解模型。MenuGen 的部署痛苦是一个缩影——当 Agent 可以写代码却无法部署代码时，人类就成为了自动化的瓶颈。Andrej 的终极测试是：给 Agent 一个提示词，能否从零部署 MenuGen 而无需人工干预？

协作层面，Andrej 展望了 Agent 间直接对话的未来。每个人都有 Agent 代表自己，每个组织也有 Agent 代表其流程。日常协调由 Agent 在毫秒级完成，人类只在关键决策点介入。这要求新的身份授权、协商协议和信任边界机制。

教育层面，Andrej 引用“可以外包思考，不能外包理解”作为核心原则。AI 可以替代计算、记忆和具体执行，但无法替代人类对问题本质的把握。他分享了自己用 LLM 知识库和合成数据生成来增强理解的工作流，强调细节可以外包给 Agent，但原理和品味必须内化为自己的能力。

这三个议题共同指向一个结论：Agent 时代的赢家不是那些“最会用 AI 做事”的人，而是那些

⁰ 视频画面时间区间：00:28:05–00:28:20。

“最懂得该让 AI 做什么、以及为什么值得做”的人。理解力，而非执行力，将成为人类最后的护城河。

6 总结与延伸

6.1 核心论点回顾：三个范式、两层编程、一个未来

Andrej Karpathy 的这场访谈，以他标志性的坦诚与深度，勾勒了一幅 AI 时代软件开发的完整图景。贯穿全篇的核心脉络可以概括为：

三个范式

- **软件 1.0**：人类显式编写规则代码，直接控制计算机
- **软件 2.0**：通过数据集训练神经网络，让权重被学习
- **软件 3.0**：以提示词和上下文窗口为杠杆，让 LLM 作为解释器执行计算

两层编程

- **Vibe Coding (氛围编程)**：抬高地板，让每个人都能创造软件
- **Agentic Engineering (智能体工程)**：保住并推高质量天花板，确保专业软件的可靠性

一个未来

不是人类辅助 AI，而是人类辅助 AI 系统。人类的角色从”做”转向”知道”——知道要建什么、为什么值得建、以及如何指挥 Agent 去建。

6.2 行动建议：今天就开始的 5 件事

基于 Karpathy 的完整论述，我们提炼出以下可立即执行的行动建议：

1. **投资你的 Agent 工具链**：像过去投资 Vim/VS Code 一样，深度定制 Claude Code、Codex 等工具。AI 原生不是天赋，而是选择。
2. **练习规格设计 (Spec)**：这是与 Agent 协作的核心技能。好的规格要足够详细以避免常识性错误，又要足够高层以给 Agent 留下执行空间。
3. **保持对底层原理的理解**：API 细节可以外包给 Agent，但张量视图 vs 拷贝、算法复杂度、架构模式等底层原理必须掌握。
4. **对 Agent 保持健康的怀疑**：记住 Agent 是”实习生”而非”同事”。它们会在你意想不到的地方犯错，需要人类的监督和验证。
5. **培养品味和判断力**：这是人类在 AI 时代不可替代的最后堡垒。审美、架构设计和需求理解不在 RL 训练目标中，因此短期内不会自动化。

6.3 开放问题：品味会自动化吗？理解会被替代吗？

Karpathy 的访谈在给出清晰判断的同时，也留下了几个值得持续思考的开放问题：

尚未回答的问题

- **品味会自动化吗？** Karpathy 的回答是” 希望会，但目前没有”。因为审美不在 RL 奖励函数中。但未来是否会出现以” 人类偏好” 为奖励信号的 RLHF 变体，让模型习得品味？
- **理解会被替代吗？** LLM 擅长基于统计模式生成合理输出，但” 真正的理解”——因果推理、抽象泛化、价值判断——仍然是人类独有的领地。这种独特性会持续多久？
- **Agent-native 基础设施何时到来？** 当前整个数字世界的基础设施都是为人类设计的。从” 为人类写文档” 到” 为 Agent 写接口” 的范式转变，需要多长时间？
- **教育应该如何改革？** 当” 做题” 可以被外包时，教育的核心目标应该是什么？Karpathy 的答案是” 理解力”，但如何系统性地培养理解力，而非仅仅传授知识？

6.4 终极测试

Karpathy 在访谈结束时提出了一个发人深省的终极测试：

” 能否给 Agent 一个提示词，让它从零部署 MenuGen 而无需人工干预？ ”

这个测试直指 Agent-native 基础设施的核心命题。当前答案是” 不能”——因为 Vercel 的 DNS 配置、Stripe 的支付设置、Google OAuth 的认证流程，每一步都需要人类在 GUI 中点击和判断。但当基础设施真正为 Agent 设计时，这个答案可能会变成” 能”。

人类成为瓶颈

在 Agent 时代，知道要建什么、为什么值得建、以及如何指挥 Agent 去构建——这三项能力都根植于**理解**，而非记忆或计算。如果你放弃了理解的追求，你就放弃了自己在 AI 时代的核心角色。

后记： 本笔记基于 Andrej Karpathy 于 2026 年 4 月 29 日在 Sequoia Capital AI Ascent 活动上的访谈整理。访谈原文时长 29 分 49 秒，涵盖从个人体验到哲学反思的完整光谱。笔记力求在保留 Karpathy 原意的基础上，以教学化的方式重构其论证结构，帮助读者系统性地理解这场正在发生的范式跃迁。